Фаза доделе ресурса

- Ова фаза се у литератури назива и фазом доделе регистара, при чему се под регистрима подразумева скуп ресурса истог типа.



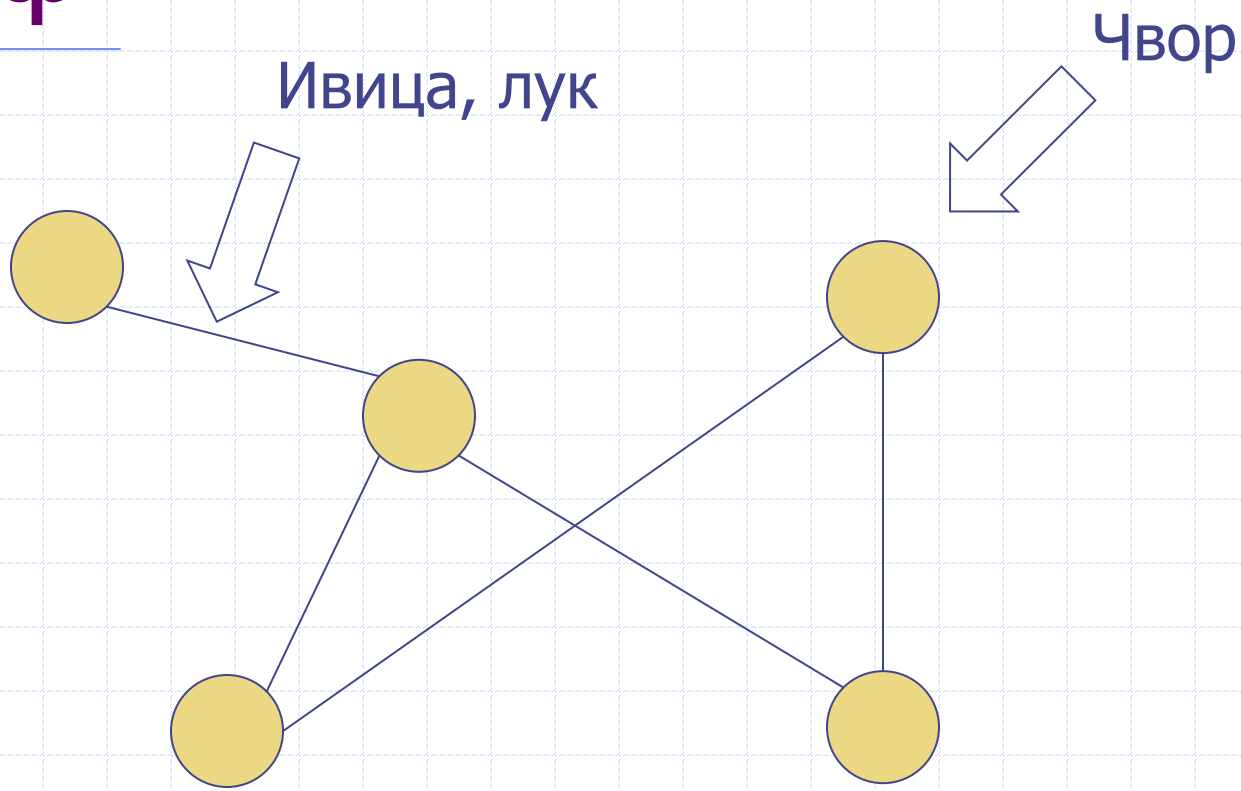
1. Са колико најмање боја се могу обојити земље на овој карти, али тако да никоје суседне две не буду обојене истом бојом?

2. Како обојити карту са тим бројем боја?



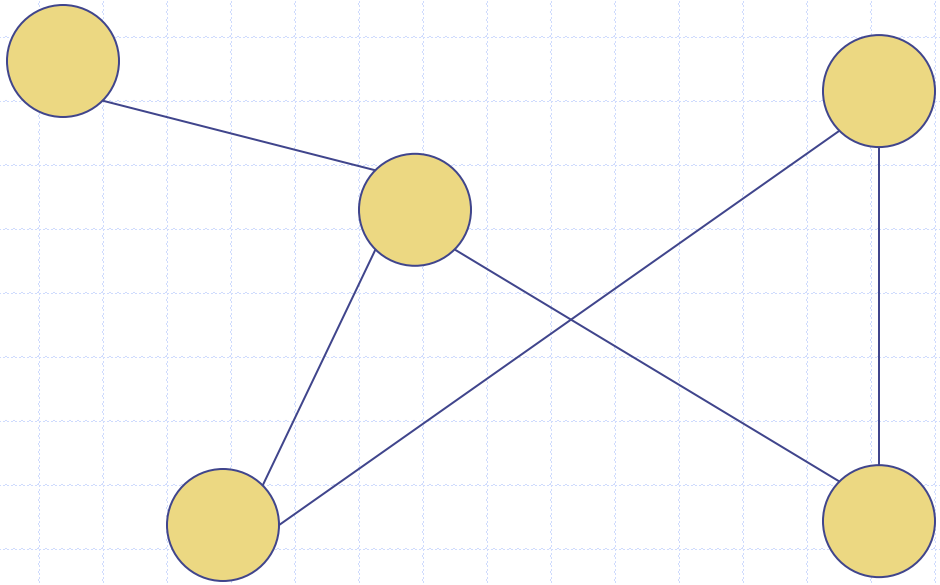
1. Довољне су четири боје.
2. ??? Није тривијално.

Граф



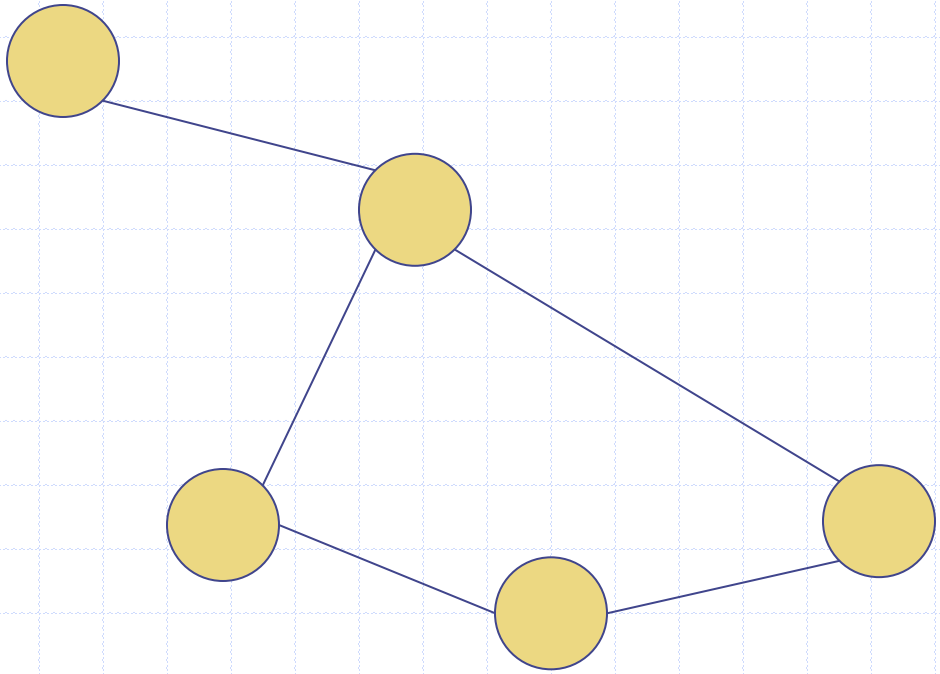
Чворови су **суседни**, ако су повезани ивицом.
Ранг (степен) чвора је број његових суседа.
Ранг (степен) графа је једнак рангу чвора са највећим рангом.

Граф



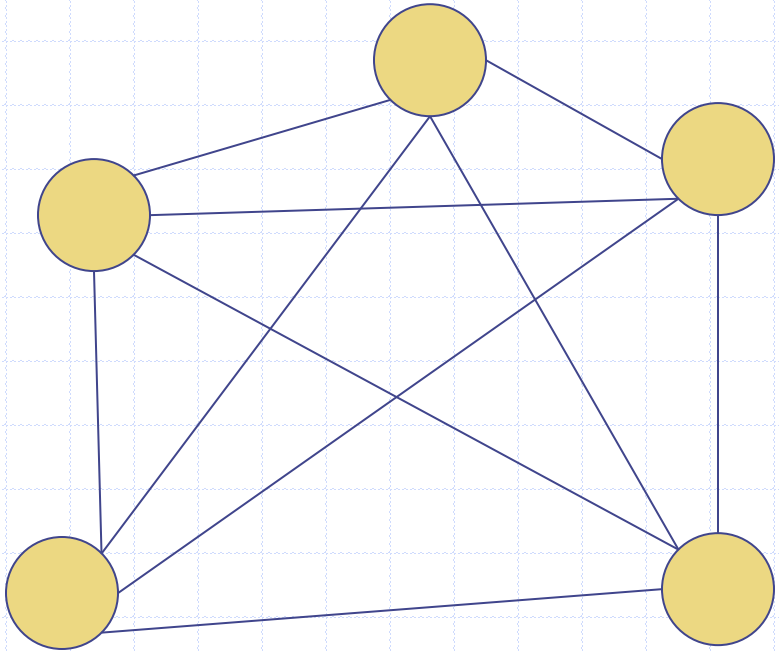
Граф је **равански** (**планаран**), ако се може сместити у раван тако да се никоје две ивице не секу.

Граф



Граф је **равански** (**планаран**), ако се може сместити у раван тако да се никоје две ивице не секу.

Граф



Клика је део чворова графа који су сви међусобно повезани.
Граф је **потпун**, ако ако сви његови чворови образују клику.

Бојење графа

Бојење графа (чворова графа) је придруживање сваком чвору неке боје, али тако да суседни чворови не могу имати исту боју.

Хроматски број графа је најмањи број боја који је довољан да се граф обоји.

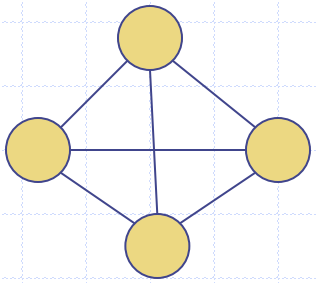
Неке границе хроматског броја:
- За равански граф, увек је довољно 4 боје.
- Увек је већи или једнак 1 и мањи или једнак броју чворова.
- Већи је или једнак броју чворова у највећој клики у графу.
- Мањи је или једнак рангу графа...

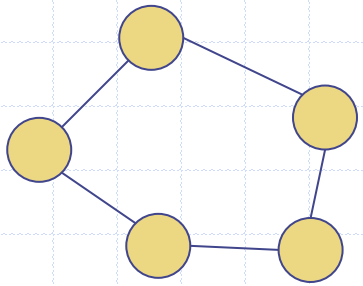
Бојење графа

Бојење графа (чворова графа) је придруживање сваком чвору неке боје, али тако да суседни чворови не могу имати исту боју.

Хроматски број графа је најмањи број боја који је довољан да се граф обоји.

Неке границе хроматског броја:
- За равански граф, увек је довољно 4 боје.
- Увек је већи или једнак 1 и мањи или једнак броју чворова.
- Већи је или једнак броју чворова у највећој клики у графу.
- Мањи је или једнак рангу графа...



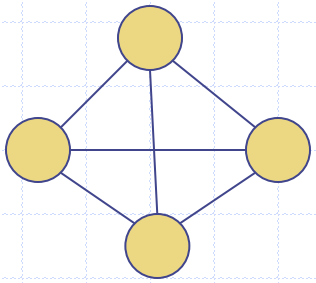


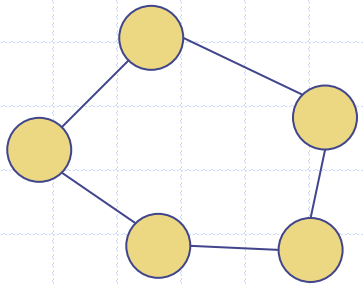
Бојење графа

Бојење графа (чворова графа) је придруживање сваком чвору неке боје, али тако да суседни чворови не могу имати исту боју.

Хроматски број графа је најмањи број боја који је довољан да се граф обоји.

Неке границе хроматског броја:
- За равански граф, увек је довољно 4 боје.
- Увек је већи или једнак 1 и мањи или једнак броју чворова.
- Већи је или једнак броју чворова у највећој клики у графу.
- Мањи је или једнак рангу графа, осим у случајевима када је граф потпун или кружан за непарним бројем чворова, када је једнак рангу + 1.

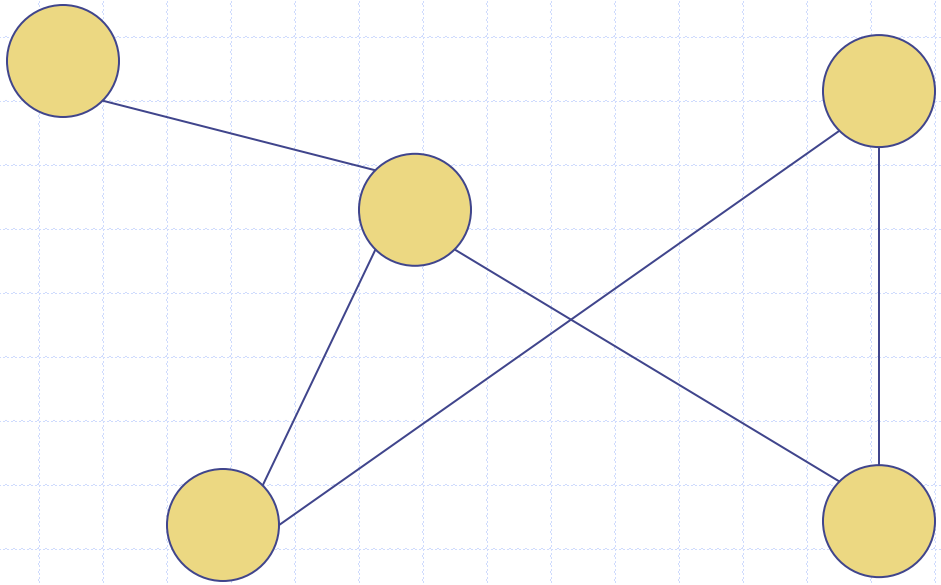






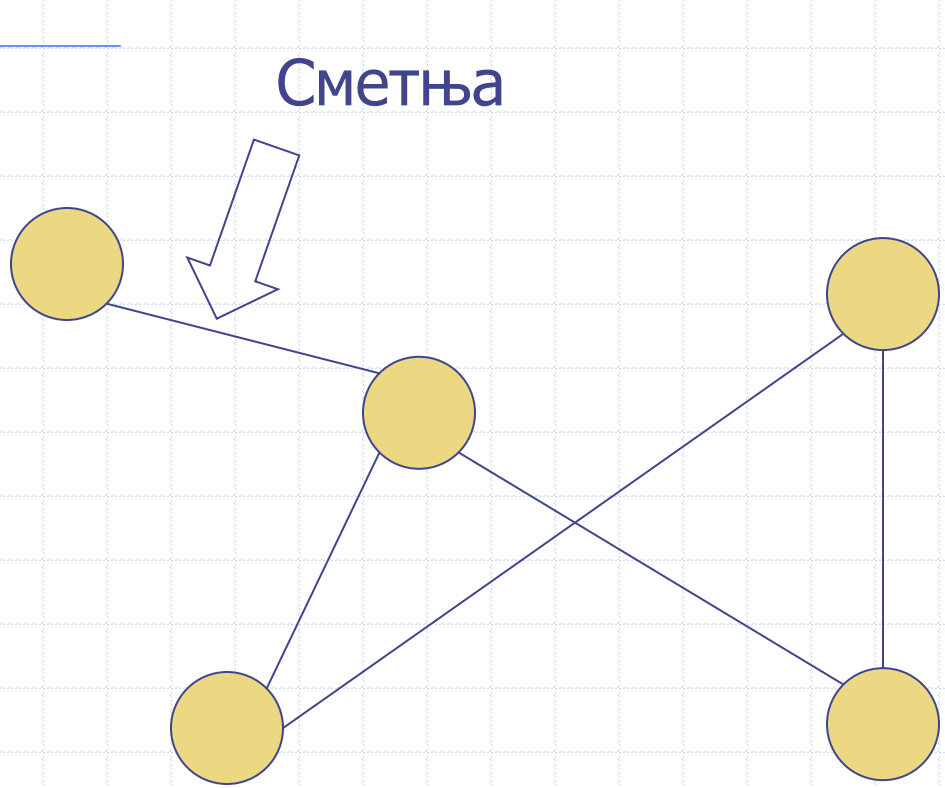
1. Са колико најмање боја се могу обојити земље на овој карти, али тако да никоје суседне две не буду обојене истом бојом?

2. Како обојити карту са тим бројем боја?



1. Са колико најмање боја се могу обојити чворови на овом графу, али тако да никоја суседна два не буду обојена истом бојом?

2. Како обојити граф са тим бројем боја?



Боје - регистри

1. Да ли се могу са К боја обојити чворови на овом графу, али тако да никоја суседна два не буду обојени истом бојом?

2. Како обојити граф са К боја?

- Имамо граф сметњи.
- Имамо К регистара.

- Два случаја:
- Хроматски број графа је мањи или једнак К
  - Наш алгоритам може да обоји дати граф
  - Наш алгоритам не може да обоји дати граф
- Хроматски број графа је већи од К

Бојење графа сметњи

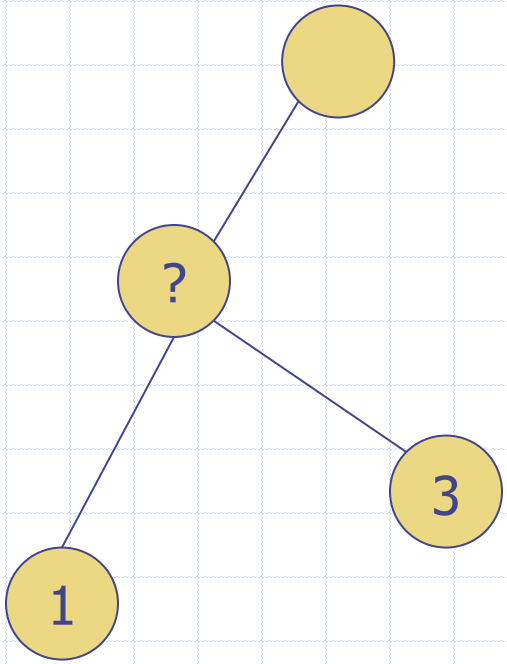
- Сваком регистру одредишног процесора одговара једна боја, тако да се код процесора са К регистара, граф сметњи боји са К боја, што се назива *К-бојење*.
- Ако је К-бојење неког графа сметњи успешно, оно одређује исправну доделу регистара операндима.
- у супротном случају, неки операнди се морају сместити у неку другу класу ресурса (нпр. меморијске локације), што се назива преливањем (енгл. spill).

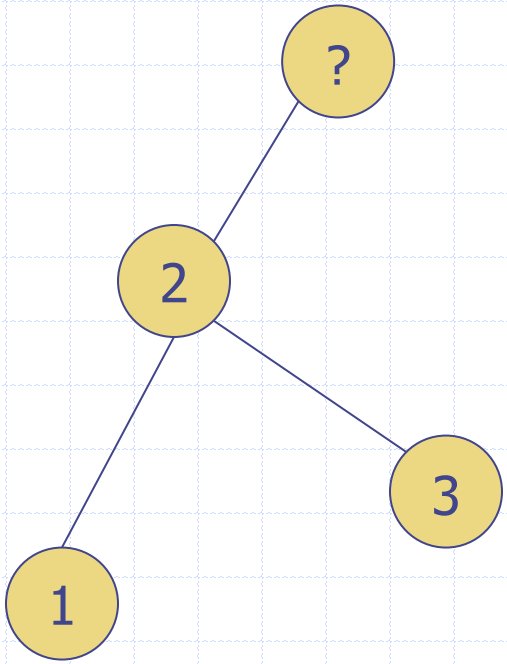
Бојење графа сметњи

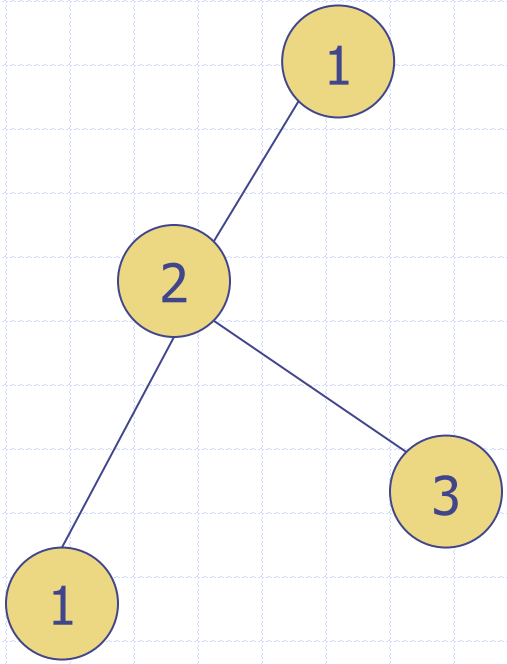
- Проблем бојења графа је сам по себи сложен.
- Проблем доделе ресурса је још сложенији, јер је бојење графа само један његов део.
- Не значи да је мање боја увек боље.
- У већини случајева, боље је да се обоји без преливања него да буде преливања... али чак ни то не мора увек бити тачно.

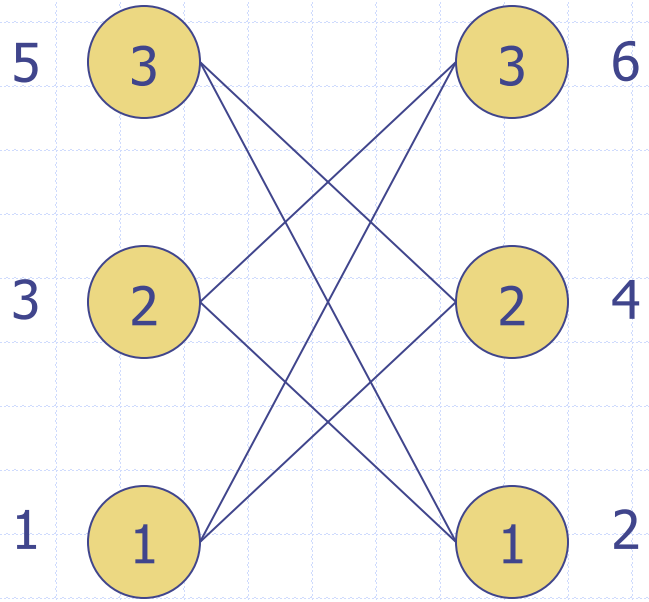
Бојење графа сметњи

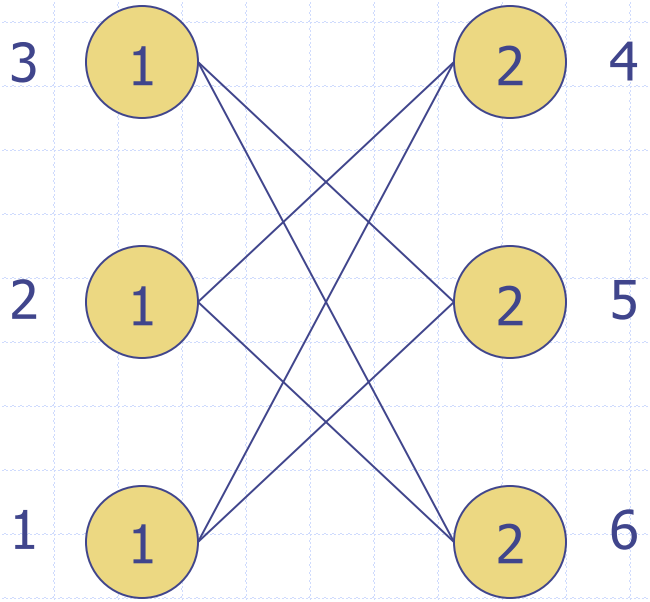
- Велика класа алгоритама се своди на пролазак кроз сваки чвор графа неким редоследом, и сваком чвору се додељује „најмања“ слободна боја (тј. која није већ заузета од стране суседних чворова).
- Највеће питање је којим редоследом ићи по чворовима.


?
3
1


?
2
3
1


1
2
3
1


5
3
1
3
2
1
3
2
1
6
4
2


3
2
1
1
1
2
4
5
6

Ранг чвора и Значајни ранг

- Деф.: Ранг чвора који је већи или једнак К (бр. расположивих боја) је *значајан ранг*.
- Деф. За чвор чији ранг је значајан каже се да је *значајан чвор*.

Основне примитиве (фазе) алгоритма за доделу ресурса (1/2)

- ФОРМИРАЈ: формира граф сметњи.
- УПРОСТИ: Чвор који није значајан се извлачи из графа и гура на стек (он није кандидат за преливање).
- ПРЕЛИЈ: Примењује се у ситуацији кад су сви преостали чворови значајни. Изабере се један значајан чвор, означи се као кандидата за преливање, извуче се из графа и гурне на стек.
- ИЗАБЕРИ: 1) Додељује боју чвору са стека, који се потом скида са стека и враћа у граф који се постепено реконструише.

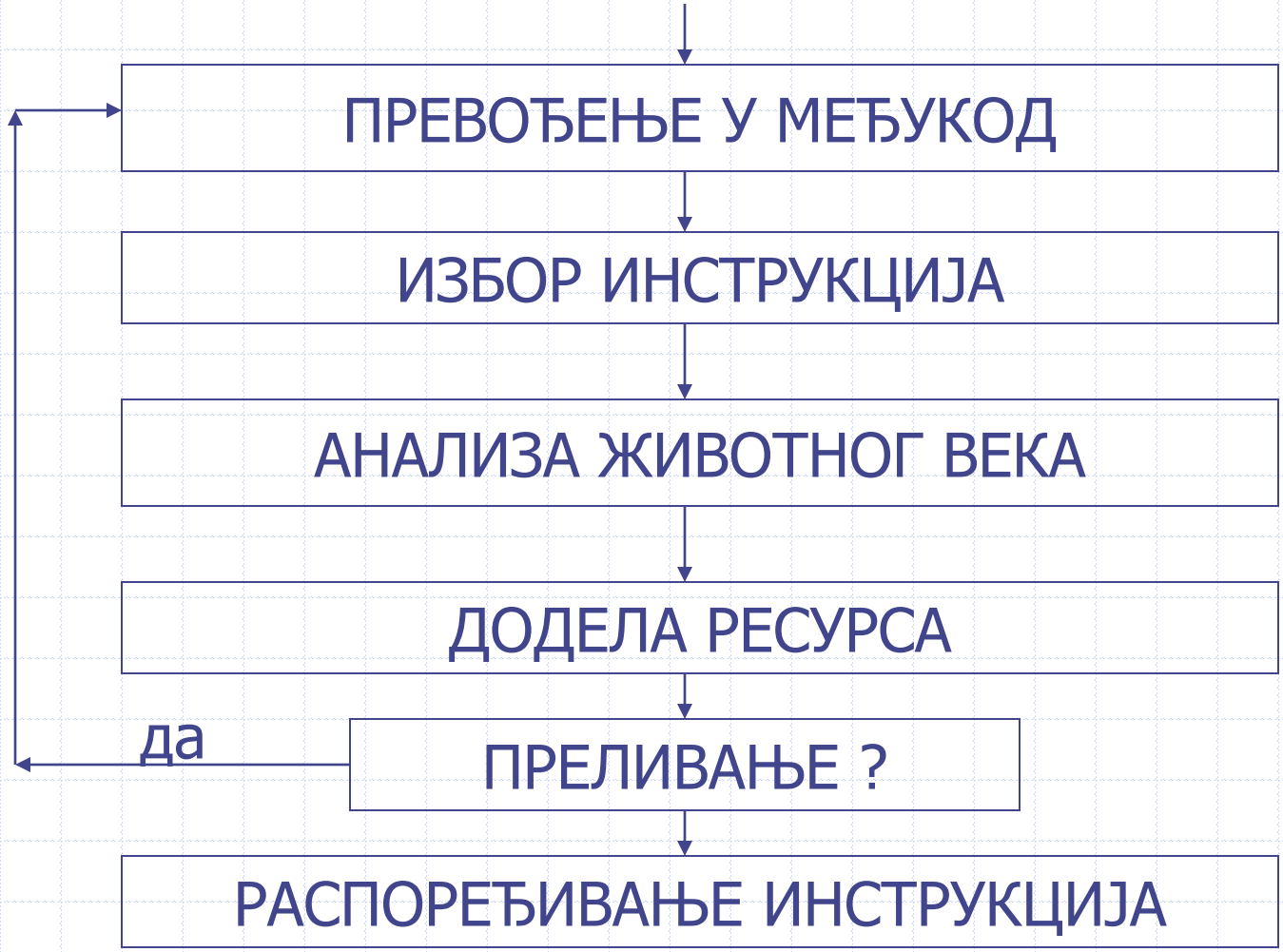
Основне примитиве (фазе) алгоритма за доделу ресурса (2/2)

- ИЗАБЕРИ: 2) Уколико нема боје, чвор се прелива.
- ПОНОВИ : Уколико је дошло до преливања, чвор ће бити смештен у ресурс друге класе (нпр. меморија). Тада се полазни програм мора модификовати додавањем инструкција:
  - За дохватање променљиве из одговарајуће мем. лок. пре сваке употребе променљиве.
  - За ажурирање одговарајуће мем. лок. након сваке дефиниције променљиве.

Аномалије примитиве ПОНОВИ

- Преливена променљива се претвара у више привремених променљивих са врло кратким опсегом живота.
- Ове привремене променљиве изазивају нове сметње са постојећим променљивама у графу сметњи.
- Због овога, као и због потребе да се додају нове инструкције, се цео поступак избора инструкција и доделе ресурса се мора поновити на новој верзији програма.

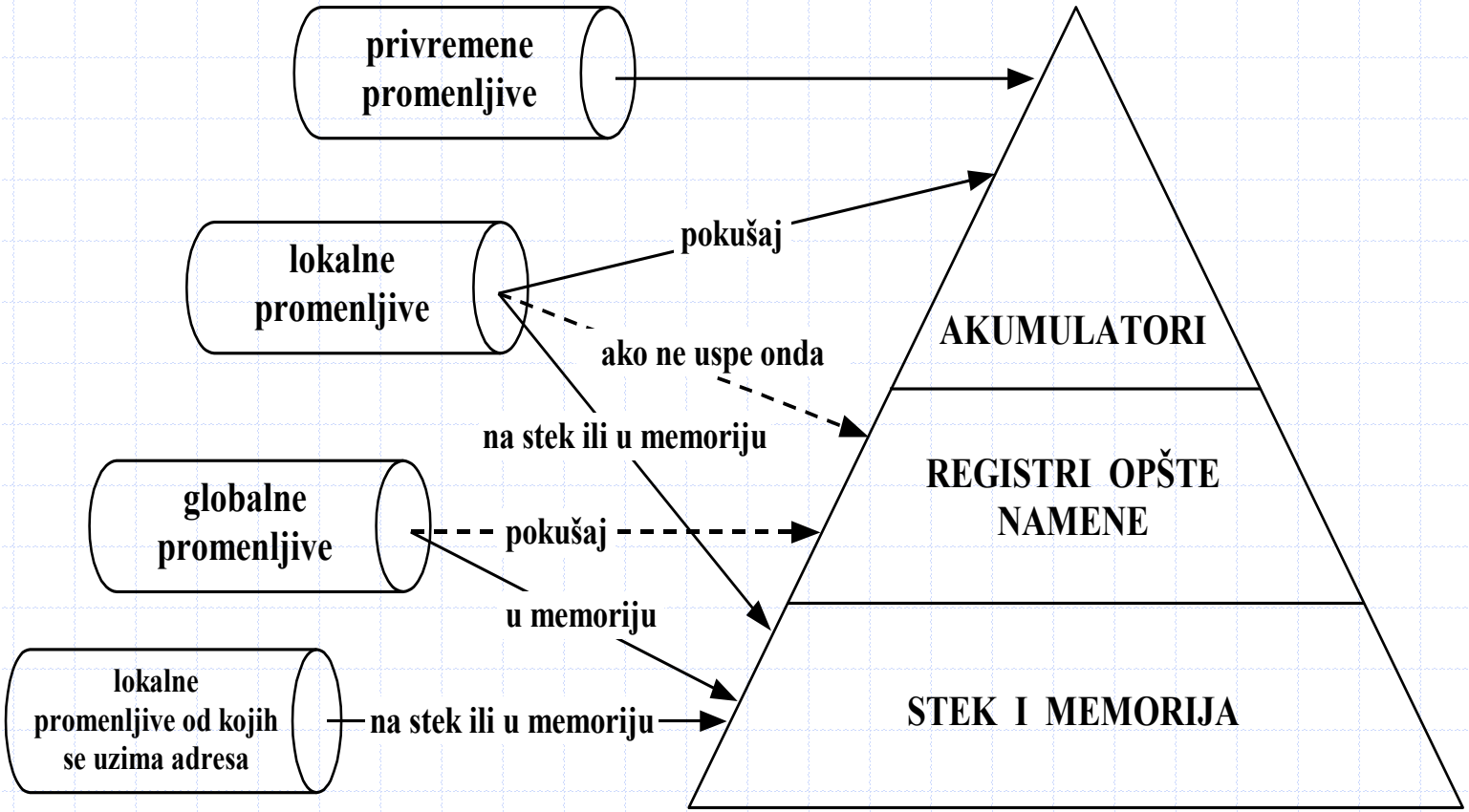
Примитива ПОНОВИ доводи до измене стабла међукода



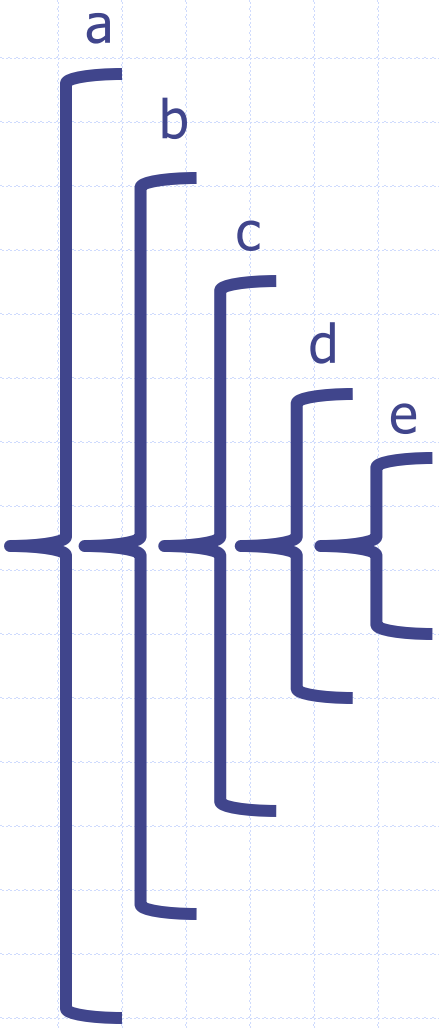
Преливање

- Што мање преливања то боље.
- Које променљиве прелити и колико њих?
- Ако прелијемо више променљивих, или променљиве са већим рангом, допринећемо бржем долажењу до обојивог графа (тј. биће мање понављања у великој петљи са претходног слајда)
- Али, тако ћемо можда прелити више него што морамо.

Пирамида ресурса

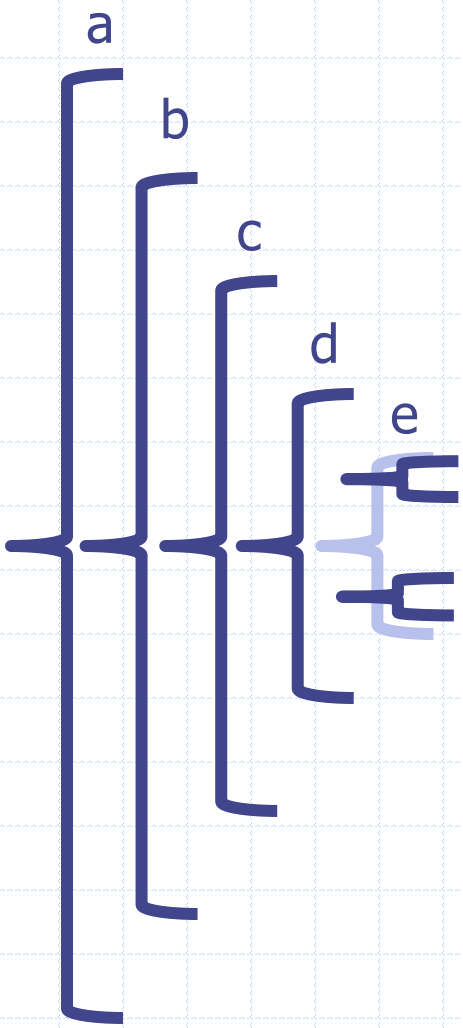


Једна илустрација



Који операнд би требало прелити?

Једна илустрација



e не смањује ранг графа

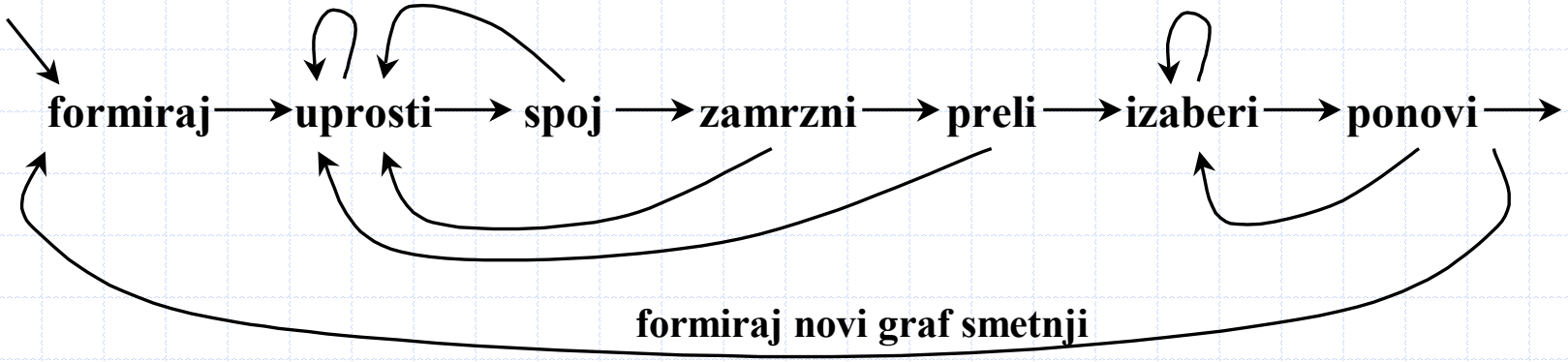
Уклањање непотребних MOVE инструкција

- Граф сметњи се користи и за уклањање непотребних MOVE инструкција.
- Уколико између изворишта и одредишта MOVE инструкције не постоји лук графа сметњи, MOVE инструкција се може уклонити, а одговарајући чворови спојити у нови чвор чији лукови су унија лукова чворова које замењује.
- Од интереса су сигурне стратегије употребе операције спајања, које неће довести до тога да обојив граф постане необојив.

Сигурне стратегије спајања чворова графа сметњи

- Стратегија Бригса (Briggs). Чворови a и b се могу спојити уколико резултантни чвор ab има мање од K значајних суседа.
- Стратегија Џорџа (George). Чворови a и b се могу спојити уколико сваки сусед чвора a има сметњу са b, или нема значајан ранг (тј. ранг му је мањи од K).

Комбинован алгоритам за доделу рег. и уклањање MOVE инст.



Модификована примитива УПРОСТИ и нове примитиве СПОЈ и ЗАМРЗНИ

- УПРОСТИ: Један по један уклањај чворове који нису у вези са MOVE инструкцијом и немају значајан ранг.
- СПОЈ: Обавља конзервативно спајање чворова.
- ЗАМРЗНИ: Уколико се ни УПРОСТИ ни СПОЈ не могу применити, тражи се чвор у вези са MOVE који нема значајан ранг. Ако он постоји, сви MOVE у вези с њим се замрзавају (одустаје се од намере да се они елиминишу спајањем чворова путем СПОЈ).

Бојење графа са спајањем чворова се обавља и за преливене

- Посматрајмо a<-b. Ако су a и b преливени, овај MOVE се реализује са t<-M[b]; M[a]<-t.
- Нови операнд t може довести до преливања још неког операнда.
- Бојење графа са спајањем преливених чворова у оквиру примитиве ПОНОВИ сада може бити агресивно.

Агресивно Бојење графа са спајањем преливених чворова

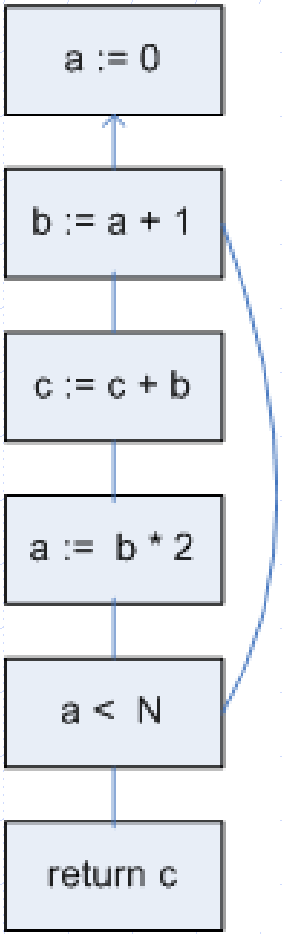
- Конструисати граф сметњи за преливене чворове.
- Спој парове преливених повезаних са MOVE који немају међусобне сметње.
- Примени УПРОСТИ и ИЗАБЕРИ – додељене боје сада одговарају меморијским локацијама преливених чворова.

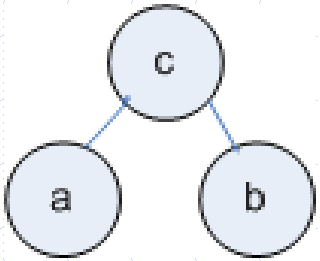
Претходно обојени операнди

- Тј. чворови са предефинисаном бојом
- Представљају машинске регистре са посебном наменом нпр. показивач стека, итд.
- Ако нису експлицитно коришћени, они не сметају обичним чворовима.
- Ако јесу експлицитно коришћени, не могу бити предмет операција УПРОСТИ и ПРЕЛИ.
- Уместо да се стално држи у претходно обојеном операнду, вредност се може пребацити у обични привремени операнд.

Пример (1/3)

Пример из анализе животног века; K=2

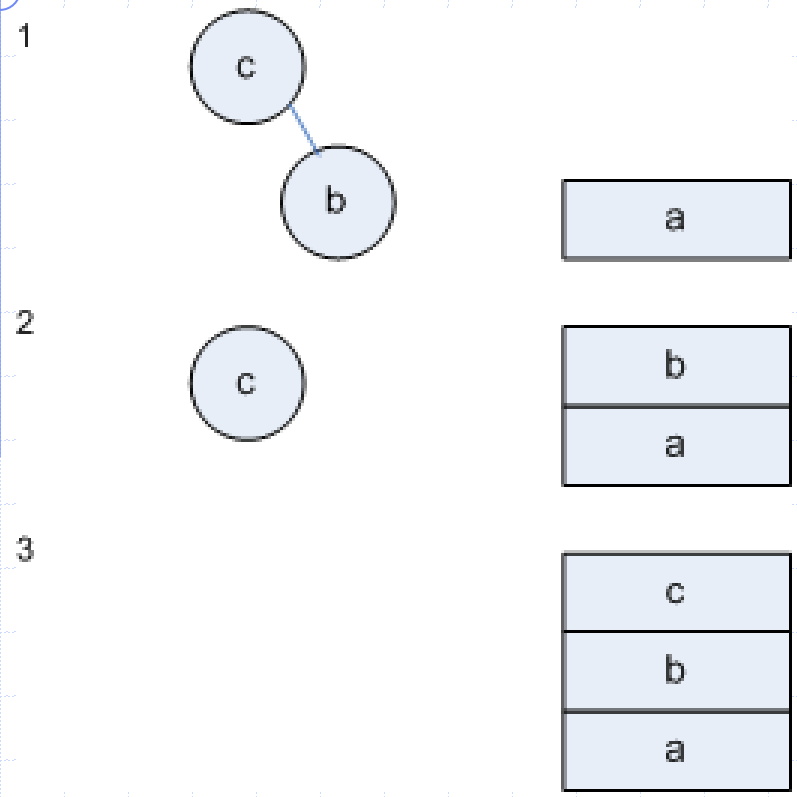




- Улаз:
  - Граф тока управљања
  - Граф сметњи

Пример (2/3)

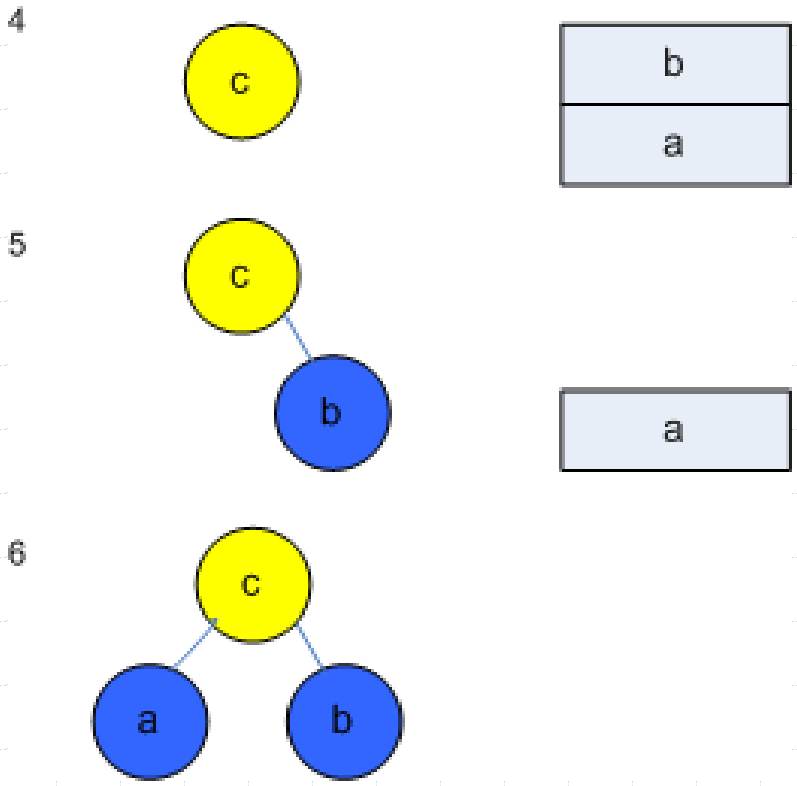
Пример из анализе животног века; K=2



1. 1. УПРОСТИ: a иде на стек (b и ц остају у графу)
2. 2. УПРОСТИ: b иде на стек (c остаје у графу)
3. 3. УПРОСТИ: c иде на стек (граф остаје празан)

Пример (3/3)

Пример из анализе животног века; K=2



4. 4. ИЗАБЕРИ: c добија жуту боју (a и b остају на стеку)
5. 5. ИЗАБЕРИ: b добија плаву боју јер има сметњу са c (a остаје на стеку)
6. 6. ИЗАБЕРИ: a добија такође плаву боју јер нема сметњу са b (стек је испражњен)

Фаза преименовања регистара

- Фаза доделе ресурса ради на два нивоа, на нивоу функције и на нивоу модула.
- На нивоу функције се додељују акумулатори и регистри опште намене за параметре функција, стек и регистар повратне адресе.
- На нивоу модула се додељују преостали регистри опште намене и меморија.

Проблем

- С обзиром да се додела ресурса на нивоу функције обавља за сваку функцију понаособ, долази до појаве да се увек исти ресурси користе у свим функцијама.
- Непосредна последица тога је да све функције увек користе акумулаторе са почетка листе акумулатора 0, 1, 2, ..., итд., односно акумулатори са краја листе се ређе користе.

Проблем - пример

```
void foo(int x);

g = нешто; // g се поставља
...
foo(5);
...
нешто = g; // g се користи
```

```
_foo:
   ...
   r1 <- 8
   ...
```

Неком операнду у foo је такође додељен регистар r1

```
r1 <- нешто
...
r0 <- 5
call _foo
...
нешто <- r1
```

Операнду g је додељен регистар r1

Два решења:

1. Променити код тако да се вредност у r1 сачува у меморију пре позива foo, и врати након тога. (Ово се може урадити у позивајућој функцији, али и на почетку и крају позване функције)
2. Преименовати ресурсе у foo тако да се уместо r1 користи нпр. r2, који позивни код не користи.

Улаз у фазу преименовања:

- граф позива функција, који се формира у фази доделе ресурса,
- информација о коришћењу акумулатора у појединим функцијама, која се такође скупља у фази доделе ресурса, и
- информација о акумулаторима који живе преко позива функција.

Излаз из фазе преименовања:

- Излаз ове фазе превођења програма је скуп табела за преименовање акумулатора.
- Концепцијски, табела за преименовање акумулатора садржи врсте за акумулаторе чије име треба променити, и две колоне, од којих прва садржи старо име акумулатора, а друга ново име акумулатора.

Бојење графа није једини приступ

- Али јесте најуспешнији, са становишта квалитета добијеног кода.
- Међутим, ако нам је и брзина превођења битна, онда можемо изабрати и другачије приступе, ако што је „линеарно скенирање“.

Унапређења

- Тешња веза између избора инструкција и доделе ресурса је један од праваца унапређивања.
- Посебан изазов су тзв. „неортогоналне архитектуре“.